

Digital Signal Processing (EEE F434/ ECE F434)

Lab 7: DSP-HDL Introduction

Instructions:

1. Lab report to be submitted (on Nalanda) at the end of the lab hours (preferably), late submission will result in loss of marks, use the lab hours to finish the work
2. All the tasks in each lab have to be in one main file, differentiate each question by commenting. Alternatively, you can zip all .m files together, name them appropriately and upload one zip/rar file. Make one Word/pdf file with all the results corresponding to the questions
3. Plots should include x label, y label, legend, and titles
4. File name: ID_Lab7.rar or zip **and** ID_Lab7.pdf
5. If the report/code is found to be copied, zero marks will be awarded [**Ensure that you show the outputs of the task to the TAs and get them evaluated. No queries will be entertained post lab session.**]

A Linear Feedback Shift Register (LFSR) is a shift register whose input bit is a linear function of its previous state. The most common linear function is XOR. LFSRs are widely used in digital systems for generating PN sequences [discussed in the lecture in the context of CDMA], test patterns for circuits, and in cryptography.

You are tasked with designing a 4-bit LFSR in Verilog. The LFSR should be based on the characteristic polynomial $x^4 + x + 1$. This polynomial determines the feedback connections, specifically a XOR operation on the values of the first and fourth bits of the shift register. The state of the LFSR updates on the rising edge of the clock.

1. Write the Verilog code for a module named lfsr that implements this 4-bit LFSR.
2. Design a moving average filter that computes the moving average of the last 4 values of a 4-bit input signal. The filter will use the shift register to store the last 4 input values, compute the average of these values on each clock cycle, and give out as an output:

$$\text{moving_avg} = (S_0 + S_1 + S_2 + S_3)/4,$$

where S_i stands for the i^{th} bit in the 4-bit shift register module.

Note: state <= 4'b0001; // Initialize to 0001 on reset

Appendix: Module wrapper and Test bench files, for your reference

```
module LFSR (  
    input CLOCK,      // Clock signal  
    input RESET,      // Active-low reset signal  
    output reg [3:0] Q // 4-bit output  
);  
endmodule
```

Lab 7: DSP-HDL Introduction

-----Test bench-----

```
module tb_LFSR;

    reg CLOCK;      // Clock signal
    reg RESET;      // Reset signal
    wire [3:0] Q;    // Output of the LFSR

    // Instantiate the LFSR module
    LFSR uut (
        .CLOCK(CLOCK),
        .RESET(RESET),
        .Q(Q)
    );
    // Clock generation (period of 10 time units)
    always begin
        #5 CLOCK = ~CLOCK; // Toggle clock every 5 time units
    end

    // Test sequence
    initial begin
        // Initialize signals
        CLOCK = 0;
        RESET = 0;

        // Apply reset
        RESET = 1; #10;
        RESET = 0; #10;
        #5;
        // Wait for some cycles to see the output pattern
        #125;

        // Finish the simulation
        $finish;
    end
    // Display output on each clock cycle
    initial begin
        $monitor("Time = %0t, RESET = %b, Q = %b", $time, RESET, Q);
    end

endmodule
```

Lab 7: DSP-HDL Introduction

```
-----  
module moving_average (  
    input wire CLK,          // Clock signal  
    input wire RESET,        // Reset signal (active high)  
    input wire DATA_IN,      // 1-bit input data (0 or 1)  
    output reg [2:0] SUM_OUT,  // 3-bit sum of the shift register  
    output reg [3:0] MOVING_AVG // 4-bit moving average (for simulation purposes)  
);  
endmodule  
-----
```

```
-----Test bench-----  
module tb_moving_average_filter;  
  
    reg CLK;  
    reg RESET;  
    reg DATA_IN;  
  
    wire [2:0] SUM_OUT;  
    wire [31:0] MOVING_AVG;  
    // Instantiate the DUT (Design Under Test)  
    moving_average_filter DUT (  
        .CLK(CLK),  
        .RESET(RESET),  
        .DATA_IN(DATA_IN),  
        .SUM_OUT(SUM_OUT),  
        .MOVING_AVG(MOVING_AVG)  
    );  
  
    // Clock generation (50 MHz clock, period = 20ns)  
    always begin  
        #10 CLK = ~CLK; // Toggle clock every 10ns  
    end  
  
    // Stimulus generation  
    initial begin  
        // Initialize signals  
        CLK = 0;  
        RESET = 0;  
        DATA_IN = 0;  
        #10;  
        // Apply reset  
    end  
endmodule
```

Lab 7: DSP-HDL Introduction

```
$display("Applying reset...");
RESET = 1; // Assert reset
#20;
RESET = 0; // Deassert reset
$display("Reset deasserted. Starting test.");

// Test data inputs (single bit at a time)
#20 DATA_IN = 1; // Input 1
#20 DATA_IN = 1; // Input 1
#20 DATA_IN = 1; // Input 1
#20 DATA_IN = 1; // Input 1

// At this point, the shift register contains: 1111 (all ones)

// Continue
// Sending 0's to the shift register
#20 DATA_IN = 0; // Input 0
#20 DATA_IN = 0; // Input 0
#20 DATA_IN = 0; // Input 0
#20 DATA_IN = 0; // Input 0

// At this point, the shift register is all zeros: 0000
// Sending a pattern
#20 DATA_IN = 1; // Input 1
#20 DATA_IN = 0; // Input 0
#20 DATA_IN = 1; // Input 1
#20 DATA_IN = 0; // Input 0

// Observe results for 40ns
#40;
$finish;
end

// Monitor outputs
initial begin
    $monitor("Time = %0t | DATA_IN = %b | SUM_OUT = %b | MOVING_AVG = %d", $time, DATA_IN,
SUM_OUT, MOVING_AVG);
end
endmodule
```
